



Pod Execution Model

How pods ship working software as fast as possible

01

Why Pods Exist

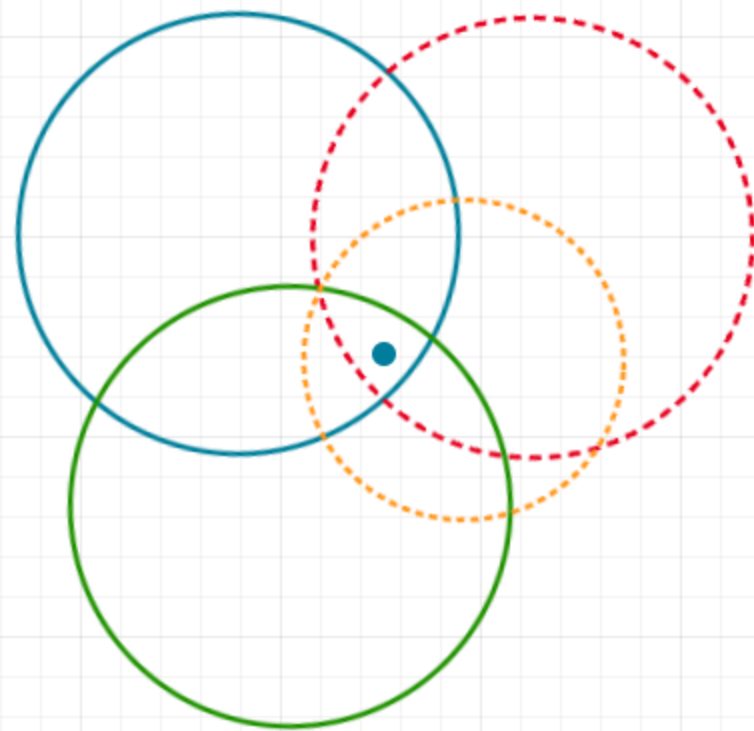
Pods compress delivery from months to weeks by replacing spec-driven waterfall with prototype-first iteration.

- Small cross-functional strike teams
- Prototype first, not spec first
- Collapse roles, remove translation layers
- AI fills the gaps that used to require more people

Calendar ships fast. That proves the model. Then it spreads.

The New Way Is Continuous

Every delivery strengthens the next. The system learns as it runs.



01 Discover

- Gather stakeholder input & user pain points
- Synthesize context from existing systems
- Align PRD with AI prompt strategy

DELIVERABLE: SIGNED-OFF PRD + PROMPT PACKAGE

02 Prototype

- Build clickable prototype same day
- Run live user feedback sessions
- Get stakeholder sign-off before code

DELIVERABLE: APPROVED PROTOTYPE + VALIDATION NOTES

03 Build

- AI-assisted rapid code generation
- API integration & data layer setup
- Continuous testing against acceptance criteria

DELIVERABLE: WORKING BUILD IN STAGING

04 Launch

- Ship to production users
- Run retrospective + capture learnings
- Feed evidence back into next Discover cycle

DELIVERABLE: RELEASE + RETRO INTELLIGENCE REPORT

Execution becomes predictable when learning never stops.

02

Pod Structure

Tiny strike team. No large teams. No handoffs. No translation layers.

⚙️ 1 Engineer

🎨 1 Designer

📄 1 PM Operator

💖 CS Partner (optional)

- Everyone prototypes. Everyone QA's. Everyone ships.
- AI is the force multiplier. Every person operates at 3–5x output

03

Inputs Replace Deliverables

The old model moved documents between roles. The new model uses AI to generate working inputs directly into a shared prototype.

The Old Way

- ✗ PM writes PRD over days, hands to Design
- ✗ Designer spends weeks in Figma, hands to Eng
- ✗ Engineer builds from spec, hands to QA
- ✗ Each step adds weeks and loses context

The New Way: Roles Collapse

- ✓ Anyone with VS Code + an approved model (Opus 4.6, Codex, Claude 5.4) can draft, design, or build
- ✓ PM drafts the problem frame AND builds a clickable prototype
- ✓ Designer drafts the PRD AND ships working HTML
- ✓ Engineer scaffolds the app AND writes the test plan
- ✓ No handoffs. Everyone contributes everywhere.

AI is why 3 people can do what 10 used to. The next four slides show exactly how.

04

PM Inputs

Write a minimal MVP PRD by feeding AI all available context. No multi-week spec cycles.

Gather Context

- Feed meeting transcriptions directly into ChatGPT
- Paste Teams conversations, support tickets, customer interviews
- Output: structured summary of what's broken and why it matters

ChatGPT

Teams

Copilot

Draft MVP PRD

- Ask AI to generate a minimal PRD from all gathered context
- Scope to one core user flow. Cut everything else.
- Output: 1-page PRD with problem, scope, constraints, and "done" definition

ChatGPT

Copilot

Build the Prototype

- PM opens VS Code + an approved model and builds a clickable prototype same day
- No designer needed for v1. Ship something testable immediately
- Output: working HTML the pod can react to, not a document to review

VS Code

Opus 4.6

Codex

What used to take a PM two weeks of writing now takes a morning with AI. Gather context, draft the PRD, move on.

05

Designer Inputs

Skip Figma. Open VS Code + an approved model. Build clickable prototypes in hours, not weeks.

Shape the Brief

- Take the PM's draft PRD and pressure-test it with UX thinking
- Feed user research and competitor screenshots into ChatGPT to surface gaps
- Output: refined brief with interaction requirements the PM might miss

ChatGPT

Copilot

Build in Code

- Open VS Code + an approved model. Describe the UI in plain language
- Full page with states, navigation, real copy. Working HTML, not static mockups
- Output: clickable prototype the pod can test same day

VS Code

Opus 4.6

Codex

Iterate Live

- Tell AI "make the sidebar narrower" or "add dark mode." See it instantly
- Refine the prototype alongside the engineer in the same codebase
- Designer becomes a creative director of AI output, not a pixel pusher

VS Code

Opus 4.6

A designer with VS Code + an approved model can produce a clickable prototype in 2 hours. That's the shift.

06

Engineer Inputs

VS Code + an approved model scaffolds the app. Engineer directs, integrates, and ensures quality.

Scaffold the Real App

- Take prototypes from PM and designer. Use AI inside VS Code, pointed at the production codebase
- Scaffold production code from prototype patterns. Bridge the gap from demo to real
- Primary user flow running end-to-end in the real app within 48 hours

VS Code

Opus 4.6

Codex

Ship and Iterate

- "Add form validation" / "add offline support."
Done in minutes, not sprints
- AI generates tests, API contracts, data models, and component variations
- Engineer's role shifts from writing code to directing AI and ensuring quality

VS Code

Opus 4.6

Codex

Own the Architecture

- Map integration surfaces: APIs, auth, data sources
- Use AI as a peer reviewer: surface risks, challenge assumptions, catch blind spots
- Deployable structure from day 1: CI, env config, feature flags

ChatGPT

VS Code

| What used to take a sprint takes 2-3 days. The engineer orchestrates AI, not writes every line.

07

QA & Leadership Inputs

QA is not a phase. It starts when the first prototype exists. Leadership removes friction, not approves output.

Embedded from Day 1

- QA joins at pod kickoff, not after engineering is "done"
- Tests the first prototype on day 1. Shapes requirements through what breaks
- Validate on prototypes, not specifications. Bugs caught in context, not after handoff
- QA supports pods, doesn't gate releases
- Uses AI to generate test scenarios, edge cases, and regression checklists from the evolving prototype

ChatGPT

VS Code

Everyone is QA

- PM validates assumptions against real user behavior, not specs
- Designer tests interactions in the working prototype, not in Figma comments
- Engineer uses AI to generate automated tests alongside every feature

VS Code

Opus 4.6

Leadership Unblocks

- Strategic framing: why this matters right now
- Fast yes/no on scope questions. No committees
- Remove organizational friction. Clear the path so the pod can ship

| There is no "hand off to QA." QA is already there. Everyone tests. Leadership clears the path.

08

Artifact Expectations

Every artifact should accelerate testing or iteration. If it doesn't, cut it.

What Pods Produce

- ✓ 1-page MVP PRD (problem, scope, done definition)
- ✓ Clickable HTML prototype (not static mockups)
- ✓ AI-generated test checklists and edge cases
- ✓ Working weekly demo for stakeholder feedback
- ✓ Short decision notes ("we chose X because Y")

What Pods Don't Produce

- ✗ Multi-page spec documents
- ✗ Polished Figma handoff files
- ✗ Status decks or approval presentations
- ✗ Documentation before the thing works
- ✗ Anything that requires a meeting to review

The artifact is the prototype. Everything else is a note that helps the prototype get better.

09

Iteration Speed

Continuous forward movement. Unfinished but operational beats polished but delayed.

- **Daily:** Something visual changes. Prototype moves forward. Pod syncs briefly on what shipped, what's next
- **Weekly:** Working demo. Testable version the team reacts to. Stakeholders give feedback on the real thing
- **Monthly:** Lessons learned sync. Pods share patterns across teams. Model validated or adjusted based on what shipped

| Speed is the strategy. The pod that ships fastest teaches the org the most.

11

Definition of Done

Done means operational, testable, usable. Not polished, finalized, documented.

Done Means

- ✓ Operational: works end to end
- ✓ Testable: someone can try it
- ✓ Adoptable: user completes the core flow
- ✓ Demonstrable: showable in a working demo

Done Does Not Mean

- ✗ Pixel perfect
- ✗ Fully documented
- ✗ All edge cases handled
- ✗ Production hardened

Pods ship capability, not perfection.

**"Teams no longer operate around a PRD.
They operate around a Prototype."**

Three Pilot Pods

One model. Three contexts. Each proves a different dimension of the new way of working.

Calendar Pod

Steve · Damion · Denny

- Ship calendar as fast as possible. Operational, not polished
- Learn what inputs are actually required to iterate fast
- This is the proving ground. If it works, the model spreads org-wide after May release
- Establishes the template for how every future team operates

Strategic Apps Pod

Brian · Sakiba · Vlado

- Strategic execution on AssetCentric Project Management
- Two app docs already exist as starting material
- Already a high-performing team. Redirect into the faster input model
- Proves the model works for strategic, not just greenfield work

Forward Deployed Pod

Jordan Hoff + engineer (TBD) + designer (TBD)

- Embed with owner customers. Visit, interview, understand the work first
- Identify high-value business processes that can be transformed with AI
- Discovery first, then implement fast. Not long drawn-out planning
- Real-world process transformation, not just internal prototyping

All three pods follow the same core rules: small teams, fast inputs, shared QA, less polish, more AI. Deliver in weeks, not months.

14

Timeline

Definition to deployment in rapid cycles. Calendar is the pilot.

- **This Week:** Staff the Forward Deployed pod (engineer + designer for Jordan). Confirm all three pod rosters
- **Next Week:** Education and alignment meeting with everyone in all three pods. Calendar pod kicks off immediately after
- **As Fast As Possible:** Calendar pod ships working product. Model validated. Team can articulate what inputs are actually needed
- **After May Release:** Pod model rolls out org-wide. Calendar pod becomes the template every new team follows

15

What Changes Now

Concrete behavior shifts starting immediately.

- No more weeks of polish before learning. Ship unfinished but operational
- PMs prototype the same day they write the PRD. Problem frames become working artifacts
- Designers move into VS Code. Layout ships as working HTML, not Figma files
- Engineers direct AI, not write every line. Scaffolding, iteration, testing all accelerated
- QA is everyone's job, embedded from day 1. No downstream handoff
- Weeks, not months. New muscle memory starts now

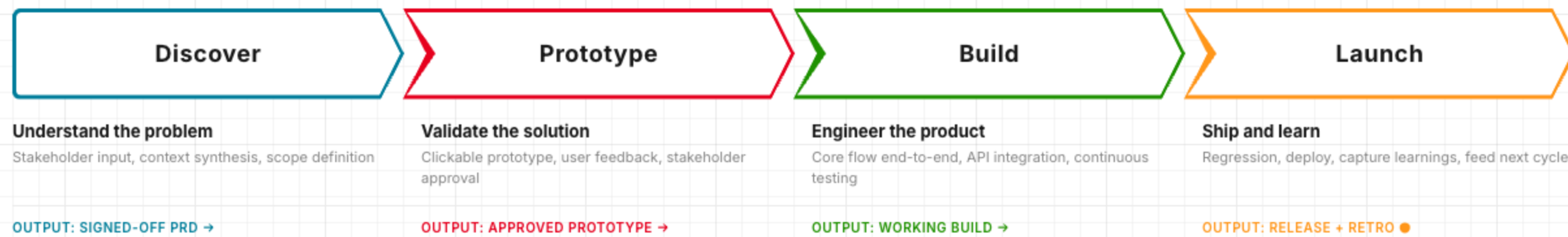
APPENDIX

Pod Execution System

17

The Flow

From problem to product. Four phases. No fixed timeline. Scale to fit.



Each phase has a clear output. The cycle runs in days or months. Same discipline.

Who Drives Each Phase

Every role is engaged every phase. Bold = peak ownership. The flow, not the calendar, determines who leads.

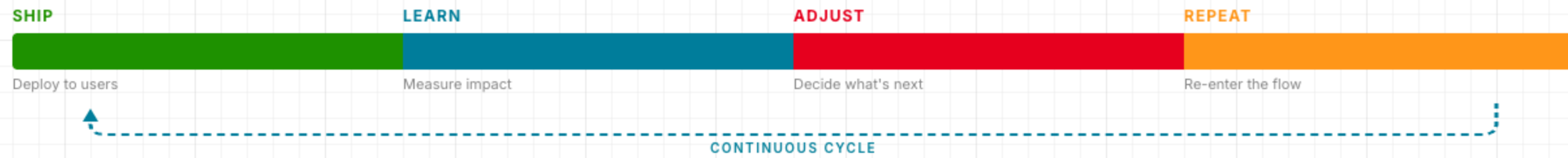
	DISCOVER	PROTOTYPE	BUILD	LAUNCH
PM	Meetings, synthesize PRD, lock scope	Build clickable prototype, validate with users	Refine scope, cut non-core, support testing	Soft launch coordination, feedback capture
Designer	Research, competitor audit, pattern library	Refine prototype, add all states and flows	Polish interactions directly in codebase	Final interaction polish, launch visuals
Engineer	Tech assessment, integration mapping, env setup	Scaffold real app from prototype patterns	Core flow end-to-end, API integration, testing	Bug fixes, edge cases, deploy, release
QA	Define test strategy from PRD	Test prototype, shape requirements through bugs	Continuous testing on real builds, edge cases	Regression pass, release sign-off
Pod Lead	Scope lock, define done, set cadence	Stakeholder demo, capture feedback, hold scope	Mid-cycle check, kill scope creep, unblock	Release decision, retrospective, capture lessons

This is not waterfall. Roles overlap. Phases compress. The grid shows peak ownership, not sequential handoffs.

19

Iteration Mechanisms

The system doesn't end at launch. Every cycle feeds the next.



After every release

Retro within 48 hours. What shipped, what slipped, what we learned.

Continuous feedback

Usage data flows directly into the next cycle. No separate research sprint.

Compounding speed

Patterns become reusable. The pod gets faster. Each cycle builds on the last.

Speed is earned, not assumed. The system compounds.